

PROJECT DOCUMENTATION

CyberSense

Interactive Training Platform
for Phishing Awareness

Web Programming

Winter Semester 2026/27

Project period: October 2026 – January/February 2027

Authors: Grace Gehlisch – ****929
Clemens Lampen – ****938
Marcel Brauns – ****503

Lecturer: Stephanie Held

Degree programme: B.Sc. Media Informatics

**HOCHSCHULE FÜR ANGEWANDTE
WISSENSCHAFTEN HAMBURG**
Faculty of Computer Science and Digital Society

Contents

1 Introduction 1

1.1 Motivation and Project Goal 1

1.2 Final Scope 1

1.3 Project Organization and Task Distribution 1

1.4 Project Timeline 2

2 System Design and Technologies 2

2.1 Technology Stack 2

2.2 Architecture and Repository Structure 2

2.3 Navigation Structure 2

2.4 Database Model 3

2.5 Scenario Design and Training Content 4

3 Implementation 4

3.1 Authentication and Authorization 4

3.2 Scenario Management 5

3.3 Training Workflow, Attempts, and Statistics 5

3.4 REST API 5

3.5 Configuration, Error Handling, and Repository Hygiene 5

3.6 Security Considerations and Limitations 6

3.7 Final User Interface 6

4 Testing and Quality Assurance 7

4.1 Automated Backend Tests 7

4.2 Manual End-to-End Verification 7

4.3 Local Setup 8

5 Future Work 8

6 AI Usage 8

7 Reflection 8

7.1 What Worked Well 8

7.2 Challenges and Lessons Learned 9

8 Conclusion 9

References 9

1 Introduction

1.1 Motivation and Project Goal

Phishing and social engineering attacks often target human decision-making rather than exploiting a purely technical vulnerability. A message can appear familiar because it imitates a known company, uses a plausible subject line, or creates a sense of urgency. Recognizing such patterns therefore requires practical training in addition to theoretical knowledge.

CyberSense was developed as a web-based training application for this purpose. Users inspect prepared e-mail scenarios and decide whether a message is **legitim** or **phishing**. The application then provides immediate feedback, an explanation, and concrete clues within the message. The main project goal is to create a complete and understandable full-stack application that combines a realistic training workflow with persistent user accounts, role-based administration, statistics, and automated backend tests.

The project does not access real mailboxes, scan attachments, or analyze live messages. All scenarios are prepared training data. This boundary keeps the project focused on the learning experience and avoids the privacy and security requirements that would arise from processing real communication.

1.2 Final Scope

The final application supports registration, login and logout, session restoration, prepared training scenarios, explanations and clues, role-based administrator access, scenario management, persistent training attempts, and personal statistics. Unknown frontend routes are handled by a custom 404 page instead of leaving the application in an undefined state.

Table 1: Functional scope of the project

Function	Status
Registration, login, logout, and session restoration	Implemented
Load and display prepared training scenarios	Implemented
Binary classification as legitimate or phishing	Implemented
Immediate feedback, explanations, and visual clues	Implemented
Persistent training attempts and personal statistics	Implemented
Role-based administrator access	Implemented
Create, edit, and delete scenarios	Implemented
Custom 404 handling	Implemented
User administration in the admin dashboard	Future work
Aggregated administrator statistics	Future work

1.3 Project Organization and Task Distribution

The project is developed collaboratively using a shared Git repository with a stable branch, a shared development branch, and feature branches for individual tasks. Core features were integrated incrementally and reviewed together before the final cleanup. Although individual areas had clear main responsibilities (Table 2), implementation, testing, debugging, and final integration were carried out collaboratively across the team.

Table 2: Main task distribution within the project team

Team member	Main responsibilities
Grace Gehlisch	Database design and implementation, SQLite integration, persistence of users, scenarios, clues and training attempts, backend integration, testing and documentation support.
Clemens Lampen	Frontend implementation, user interface design, training workflow, reusable React components, responsive layout and usability improvements.
Marcel Brauns	Backend API development, authentication and session handling, administrator functionality, REST endpoints, validation and testing support.

1.4 Project Timeline

The development process was divided into four main phases. The timeline below gives a compact overview of the progression from the initial concept to final verification and documentation.

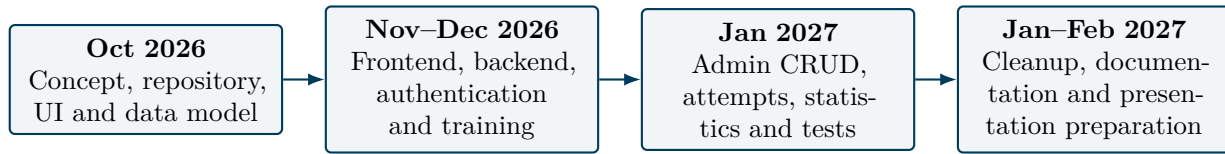


Figure 1: Project timeline from concept to finalization

2 System Design and Technologies

2.1 Technology Stack

The project uses JavaScript throughout frontend and backend. This reduces context switching and makes it easier to trace data from the React user interface through the REST API to SQLite. The final interface primarily uses custom CSS; Bootstrap is not required for the implemented visual design.

Table 3: Technology stack

Technology	Area	Purpose
JavaScript	Full stack	Main programming language
React 19	Frontend	Component-based user interface
React Router	Frontend	Client-side routing and protected routes
Vite	Frontend	Development server and production build
Custom CSS	Frontend	Responsive visual design
Node.js	Backend	JavaScript runtime
Express 5	Backend	REST API and HTTP routing
express-session	Backend	Session-based authentication
bcrypt	Security	Password hashing
SQLite / node:sqlite	Database	Persistent local data storage
Supertest	Testing	HTTP-level API tests
Node.js Test Runner	Testing	Automated test execution
Git / GitHub	Development	Version control and collaboration

2.2 Architecture and Repository Structure

CyberSense follows a client-server architecture. The React frontend never accesses SQLite directly. Instead, frontend service modules send HTTP requests to the Express backend. Routes validate input and apply authentication or authorization where required. Repository modules contain the database operations and map SQLite rows to JavaScript objects.

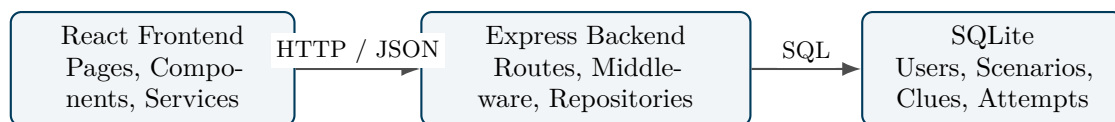


Figure 2: CyberSense client-server architecture (author's own illustration)

2.3 Navigation Structure

The application navigation separates public pages, authenticated user areas, and the administrator workflow. Figure 3 summarizes the main page transitions and shows how login and registration lead into the training and statistics areas while administrator functions remain a separate protected branch.

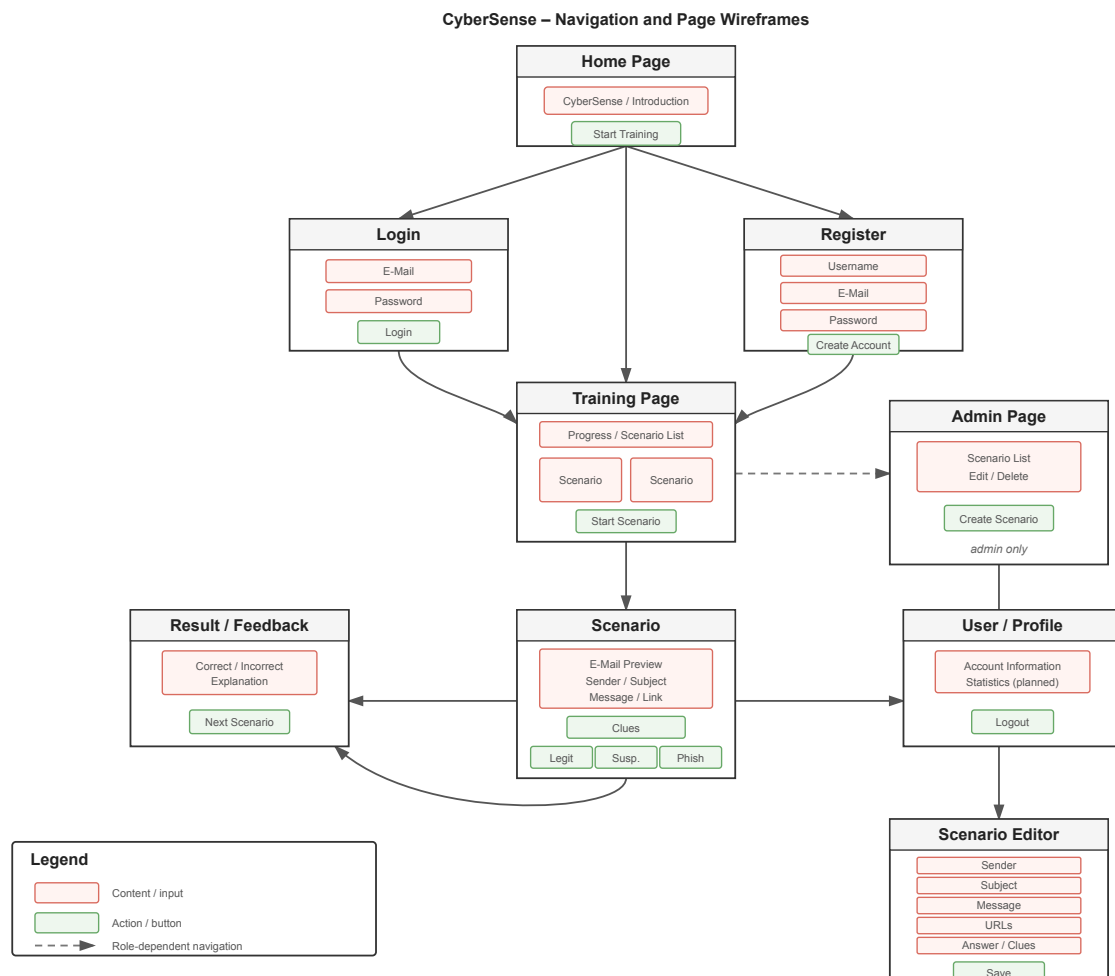


Figure 3: CyberSense navigation and page structure (author's own illustration)

The most relevant folders are shown below. Frontend configuration is centralized so that backend URLs do not have to be repeated in individual service modules.

```

CyberSense/
  backend/
    src/
      data/           prepared scenario data
      database/       SQLite initialization and seeding
      middleware/     authentication and authorization
      repositories/   database access
      routes/         REST endpoints
      test/           automated API tests

  frontend/
    src/
      components/     reusable UI components
      config/         central API configuration
      pages/          application pages
      services/       backend communication
      styles/         page-specific styling
  
```

2.4 Database Model

The final SQLite database contains four central tables: **users**, **scenarios**, **clues**, and **attempts**. User accounts contain the stored password hash and role. Scenarios contain the training message and expected classification. Clues reference concrete parts of a scenario, while attempts link a user to a scenario and record the selected answer and result.

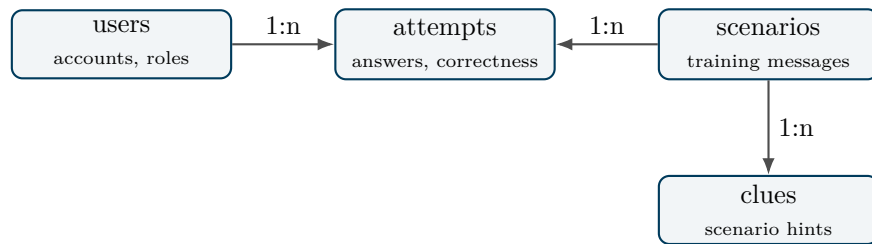


Figure 4: Simplified database relationships (author’s own illustration)

Foreign-key checks are enabled. Related clues and attempts are connected to their parent records through foreign keys. Both `correct_answer` in `scenarios` and `selected_answer` in `attempts` are restricted by SQLite `CHECK` constraints to `legitim` and `phishing`. The development and automated test environments use separate SQLite database files.

2.5 Scenario Design and Training Content

The training data is intentionally modeled as structured application data instead of static markup in the frontend. A scenario contains sender information, subject, greeting, message paragraphs, optional action text and URLs, a signature, the correct classification, and a short explanation. This structure allows the same rendering component to display both legitimate and phishing examples consistently.

Prepared scenarios imitate familiar services and common communication patterns, but training links use safe fictional or reserved domains rather than requiring interaction with real malicious infrastructure. The scenarios cover different indicators instead of relying on one repeated pattern. Examples include unusual sender domains, mismatched link destinations, unexpected payment requests, account warnings, artificial urgency, and messages that are deliberately legitimate. Legitimate examples are important because a training system containing only attacks would teach users to reject every message instead of evaluating evidence.

The clue model is separated from the main scenario so that feedback can point to more than one indicator and can be associated with a concrete area of the displayed e-mail. The renderer recognizes target areas such as sender, subject, greeting, paragraphs, links, and signature. This creates a direct connection between the explanation and the visual message that the user just evaluated.

Table 4: Examples of training indicators represented by scenarios and clues

Indicator	Training purpose
Sender address	Compare a familiar display name with the actual sender domain
Displayed vs. actual URL	Recognize links whose visible text and destination differ
Urgency or pressure	Identify language intended to reduce careful decision-making
Unexpected request	Question payments, account actions, or delivery fees without context
Legitimate message	Practice accepting plausible messages when indicators are consistent

3 Implementation

3.1 Authentication and Authorization

Registration and login are implemented by the backend rather than as frontend-only prototypes. Passwords are hashed with `bcrypt` before they are stored. After a successful login, the backend stores the user ID in an Express session. The frontend restores the current user through `GET /api/auth/me` and includes the session cookie in requests with `credentials: "include"`.

Authorization is enforced in two layers. Protected React routes prevent ordinary navigation to administrator pages, while backend middleware performs the decisive server-side check. `requireAuth` verifies that a session exists and `requireAdmin` additionally checks the user’s role. Therefore, bypassing the frontend does not grant access to protected scenario operations.

3.2 Scenario Management

Administrators can create, view, edit, and delete training scenarios. Scenario content is stored in `scenarios`; related hints are stored in `clues`. The administration form uses the fixed fictional recipient `Max Mustermann` `<max.mustermann@example.de>` so every message is shown from a consistent user perspective.

A scenario can only be classified as `legitim` or `phishing`. The previous intermediate category was removed so the training always requires a clear decision. The same two values are enforced in the frontend, backend validation, seed data, and SQLite constraints. Prepared SQL statements use placeholders rather than concatenating request values into SQL strings. Creating and updating scenario data is performed transactionally so related changes are committed together or rolled back.

Clues can target specific parts of the rendered message such as the sender, subject, greeting, individual paragraphs, a link, or the signature. This lets the training feedback explain not only that an answer was wrong, but also where the relevant indicator was located.

3.3 Training Workflow, Attempts, and Statistics

The training page loads scenarios from the REST API and presents them one after another. The user examines the message and chooses between `legitim` and `phishing`. After the decision, the frontend shows whether the answer was correct, displays the scenario explanation, and makes the configured clues visible in the e-mail preview.

For authenticated users, the frontend submits only the scenario ID and selected answer to `POST /api/attempts`. The backend loads the scenario and determines `isCorrect` itself. The user ID is also derived from the active session rather than being accepted from the client. This design prevents the browser from deciding whether its own answer should be stored as correct or from assigning an attempt to a different account.

The statistics page combines the authenticated user's stored attempts with scenario information. It shows the total number of attempts, correct and incorrect answers, success rate, completed scenarios, and recent activity. Because attempts are persisted in SQLite, the statistics remain available after page reloads and later logins.

3.4 REST API

Table 5: Relevant REST API endpoints

Method	Endpoint	Purpose
POST	<code>/api/auth/register</code>	Create account
POST	<code>/api/auth/login</code>	Authenticate and create session
GET	<code>/api/auth/me</code>	Return current authenticated user
POST	<code>/api/auth/logout</code>	End current session
GET	<code>/api/scenarios</code>	Load all scenarios
GET	<code>/api/scenarios/:id</code>	Load one scenario
POST	<code>/api/scenarios</code>	Create scenario (admin)
PUT	<code>/api/scenarios/:id</code>	Update scenario (admin)
DELETE	<code>/api/scenarios/:id</code>	Delete scenario (admin)
POST	<code>/api/attempts</code>	Store one training answer
GET	<code>/api/attempts/me</code>	Load current user's attempts

3.5 Configuration, Error Handling, and Repository Hygiene

Frontend API access is configured centrally in `frontend/src/config/api.js`. The environment variable `VITE_API_BASE_URL` can override the local fallback `http://localhost:3000/api`. A documented example is stored in `frontend/.env.example`. This removes environment-specific URLs from individual service modules.

The backend reads `SESSION_SECRET` from the environment. A development fallback exists for local work, while an explicitly configured secret is expected in a production environment. Environment files, SQLite databases, cookie files, and the Vite `dist` directory are excluded from version control. These files contain local state or generated output and are not required in the repository.

Unknown frontend routes are handled by a dedicated 404 page. Service modules also check HTTP responses and convert failed requests into errors that pages can handle. The project team does not operate the target deployment infrastructure itself; hosting is provided through the university environment. For that reason, deployment is treated as an external operating condition rather than an unfinished project feature.

3.6 Security Considerations and Limitations

Security decisions that affect authorization or stored results are implemented on the server. Frontend route guards improve usability, but they are not treated as a security boundary. Administrator API operations still require an authenticated session and the appropriate role. Similarly, the attempts API derives the user from the session and calculates correctness from the stored scenario instead of trusting values supplied by the browser.

Input handling follows the same principle. Scenario routes validate required fields and the allowed answer values before persistence. Repository operations use prepared SQL statements with placeholders. The database adds another consistency layer through foreign keys and `CHECK` constraints. These measures do not make the project a complete production security platform, but they prevent several avoidable classes of application error within the defined scope.

The session configuration is intentionally environment-aware. Local development can use a fallback secret for convenience, while the university runtime can provide `SESSION_SECRET` externally. Secrets and environment files are not stored in the repository. During final repository cleanup, local SQLite files and cookie files were also removed from version control and added to the project-level `.gitignore`. Generated frontend build output is treated in the same way.

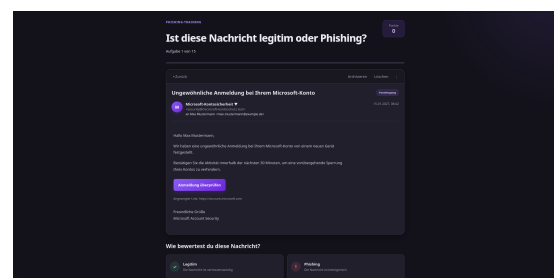
Some production-hardening topics remain outside the module scope. A public Internet deployment would require decisions about secure cookie flags behind HTTPS, a persistent session store, rate limiting, monitoring, backup strategy, operational secret rotation, and potentially stronger account recovery flows. CyberSense is therefore documented as an educational application with explicit operational limits rather than as a finished commercial security product.

3.7 Final User Interface

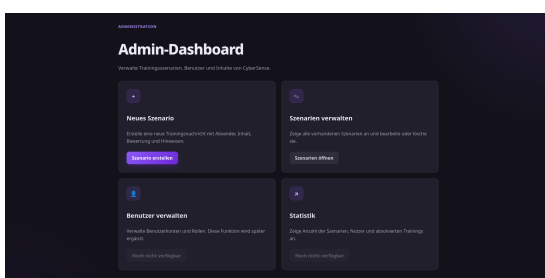
The following figure is prepared for the final screenshots. The placeholders can be replaced simply by placing the corresponding image files in the `screenshots/` directory.



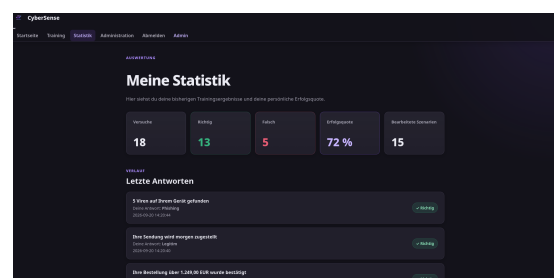
Homepage and navigation



Training scenario and feedback



Administrator scenario management



Personal training statistics

Figure 5: Final CyberSense user interface screenshots (author's own application)

4 Testing and Quality Assurance

4.1 Automated Backend Tests

The backend is tested with the Node.js Test Runner and Supertest. Tests use `cybersense-test.sqlite`, while normal development uses `cybersense.sqlite`. During development, parallel test files writing to the same SQLite database caused contention. The final test script therefore executes the suites sequentially.

```
npm test

# package.json
NODE_ENV=test node --test --test-concurrency=1
```

Table 6: Automated backend test coverage

Suite	Covered areas	Tests
Attempt API	authentication, validation, missing scenarios, correct/incorrect attempts, /me, separation of user data	9
Auth API	registration, duplicate e-mail, validation, login failures, session persistence, /me, logout	9
Scenario API	loading, authentication, admin permissions, create/update/delete, validation, clues, unknown IDs	17
Total		35

The final verified backend test run completed with 35 passing tests and no failures. The test suite is especially important because authentication, session state, role checks, and persistent database writes interact across several modules.

4.2 Manual End-to-End Verification

In addition to automated backend tests, the main user flows were checked manually in the browser. Registration, login and logout, training, persistent attempts, the statistics page, administrator access, scenario creation, editing and deletion, page reloads, and the custom 404 route were verified. The final Vite production build was also created successfully after the API configuration changes.

Table 7: Manual verification of central user flows

Area	Verification	Result
Registration	Create a new account and continue with authenticated state	Successful
Login / logout	Authenticate, restore session, and terminate session	Successful
Training	Classify multiple scenarios and inspect explanations and clues	Successful
Statistics	Reload page and verify persisted attempt counts and recent activity	Successful
Admin access	Open protected administration area with administrator role	Successful
Scenario CRUD	Create, edit, and delete a scenario through the interface	Successful
Authorization	Verify normal users cannot perform administrator operations	Successful
Routing	Open an unknown frontend route and display the custom 404 page	Successful
Production build	Run the Vite build after final configuration cleanup	Successful

A short reproducibility workflow is:

1. Start backend and frontend in separate terminals.
2. Register or log in with a normal user and complete several scenarios.
3. Confirm that results appear in personal statistics after a page reload.
4. Log in with an administrator account and create, edit, and delete a scenario.
5. Run `npm test` in the backend and `npm run build` in the frontend.

4.3 Local Setup

A clean checkout can be started with the following commands:

Table 8: Important setup and development commands

Purpose	Command
Clone repository	<code>git clone https://github.com/GraciousGames/CyberSense.git</code>
Install backend	<code>cd backend && npm install</code>
Start backend	<code>cd backend && npm run dev</code>
Install frontend	<code>cd frontend && npm install</code>
Start frontend	<code>cd frontend && npm run dev</code>
Run backend tests	<code>cd backend && npm test</code>
Create frontend build	<code>cd frontend && npm run build</code>
Backend health check	<code>curl http://localhost:3000/api/health</code>

By default, Vite serves the frontend on <http://localhost:5173> and the backend runs on <http://localhost:3000>. The SQLite database is created automatically when the backend starts and is seeded with prepared scenarios if the scenarios table is empty.

5 Future Work

The implemented scope is complete for the current module project. The administration dashboard nevertheless already visualizes two possible next steps: user management and aggregated administrator statistics. These dashboard areas are intentionally not functional in the submitted version and should be understood as a preview of future development rather than unfinished core functionality.

A future version could allow administrators to search users, change roles, or deactivate accounts. Aggregated statistics could show overall success rates, difficult scenarios, and common misclassifications without replacing the existing personal statistics. Other useful extensions include scenario categories and difficulty levels, more training content, more convenient editing of multiple clues, frontend component tests, and a dedicated accessibility audit.

Connecting CyberSense to real mailboxes would be a much larger extension. It would require a separate privacy, permissions, data retention, and security concept and is therefore deliberately outside the current project scope.

6 AI Usage

Generative AI was used as a development support tool for explaining React, Express, and SQLite concepts, interpreting error messages, discussing possible code structures, proposing test cases, adding explanatory comments, and improving documentation wording. AI-generated suggestions were reviewed and adapted before integration.

Functional behaviour was not accepted solely because it was suggested by an AI tool. Changes were checked in the running application, with API tests, or with the production build where appropriate. The final responsibility for implementation choices and the submitted project remains with the project team.

7 Reflection

7.1 What Worked Well

The separation between frontend services, Express routes, middleware, repositories, and database modules proved useful as the project grew. Authentication, administrator CRUD, attempt persistence, and statistics could be added without replacing the basic architecture. Centralizing API configuration late in the project also reduced repeated URLs and made the frontend services more consistent.

Automated tests became particularly valuable once sessions, roles, validation, and database writes interacted. The 35 passing backend tests provide a reliable regression baseline and made larger cleanups, such as reducing the answer model to two categories, safer to perform.

7.2 Challenges and Lessons Learned

Session-based authentication required the frontend to consistently include cookies in authenticated requests. Automated tests initially encountered SQLite contention when multiple test files wrote to the same test database concurrently; sequential execution provided a simple and reproducible solution for the project scope.

Another challenge was keeping the frontend forms, API payloads, repository mappings, seed data, and SQLite constraints synchronized. The removal of the former third classification showed how one seemingly small domain-model change affects every layer of a full-stack application. The final cleanup therefore included source searches, backend tests, a production frontend build, and repository hygiene checks.

For a future project, the API contract, database model, and environment configuration would be documented earlier. Automated tests would also be introduced as soon as the first state-changing endpoints are implemented instead of primarily after larger parts of the application already exist.

8 Conclusion

CyberSense demonstrates a complete full-stack workflow for phishing-awareness training. The project combines a React interface with an Express REST API and SQLite persistence. Its final scope includes account handling, session authentication, role-based administrator access, scenario CRUD, binary phishing classification, contextual clues, persistent training attempts, personal statistics, error handling, and automated backend tests.

The project also defines clear boundaries. It trains users with prepared examples rather than processing real e-mail accounts, and its two non-functional administrator dashboard areas are documented as future work. The repository structure, environment configuration, setup instructions, and test suite provide a reproducible basis for maintaining or extending the application in a later iteration.

References

References

- [1] Meta Platforms, Inc.: *React Documentation*. <https://react.dev/>
- [2] OpenJS Foundation: *Node.js Documentation*. <https://nodejs.org/docs/latest/api/>
- [3] OpenJS Foundation: *Express Documentation*. <https://expressjs.com/>
- [4] SQLite Consortium: *SQLite Documentation*. <https://www.sqlite.org/docs.html>
- [5] SuperAgent Contributors: *Supertest - HTTP assertions for Node.js*. <https://github.com/ladjs/supertest>